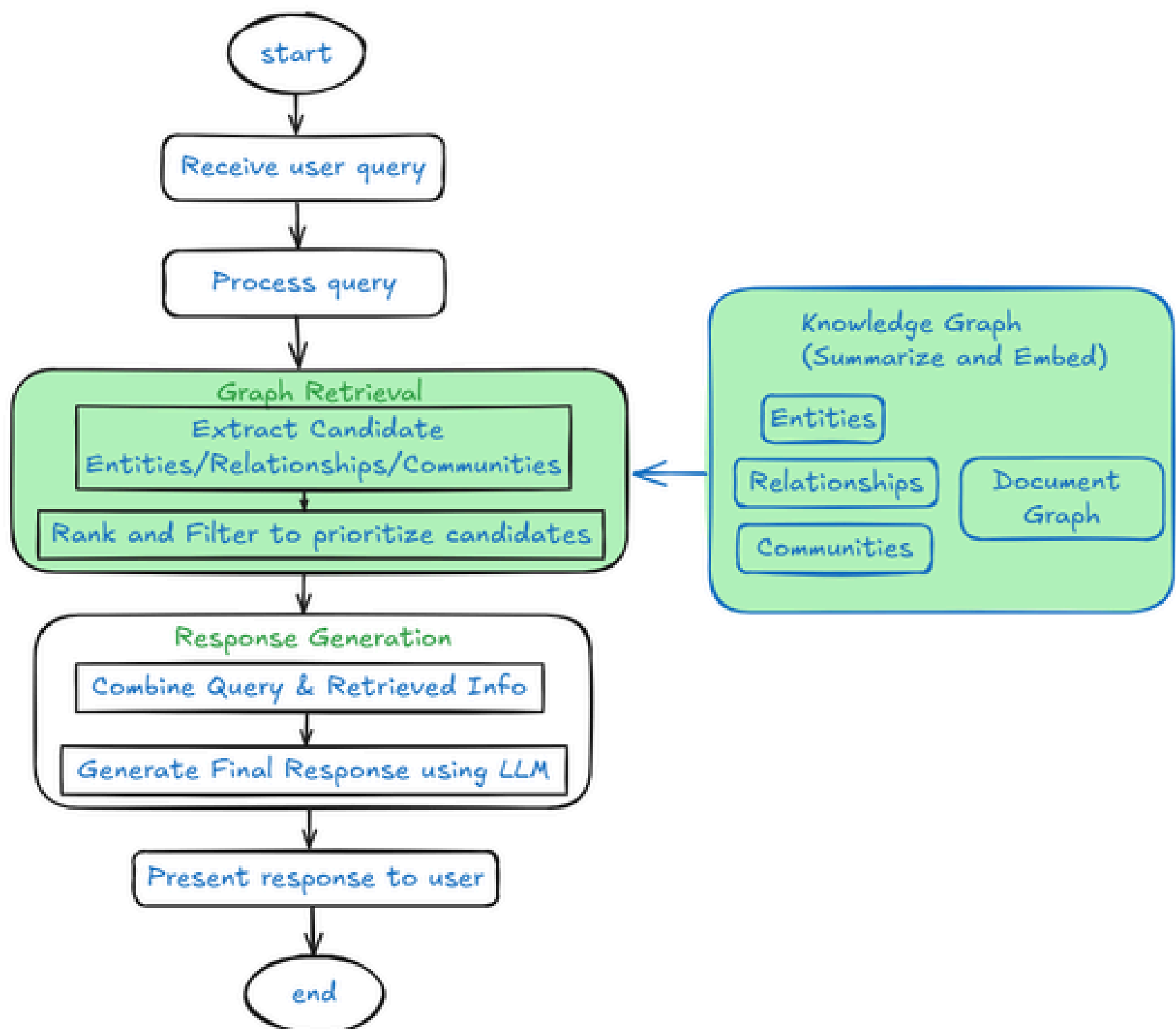


7 Steps to Build a Graph RAG

Traditional RAG retrieves chunks. Graph RAG retrieves relationships the secret to multi-hop reasoning, coherent answers, and enterprise-grade retrieval.



Where traditional RAG breaks down

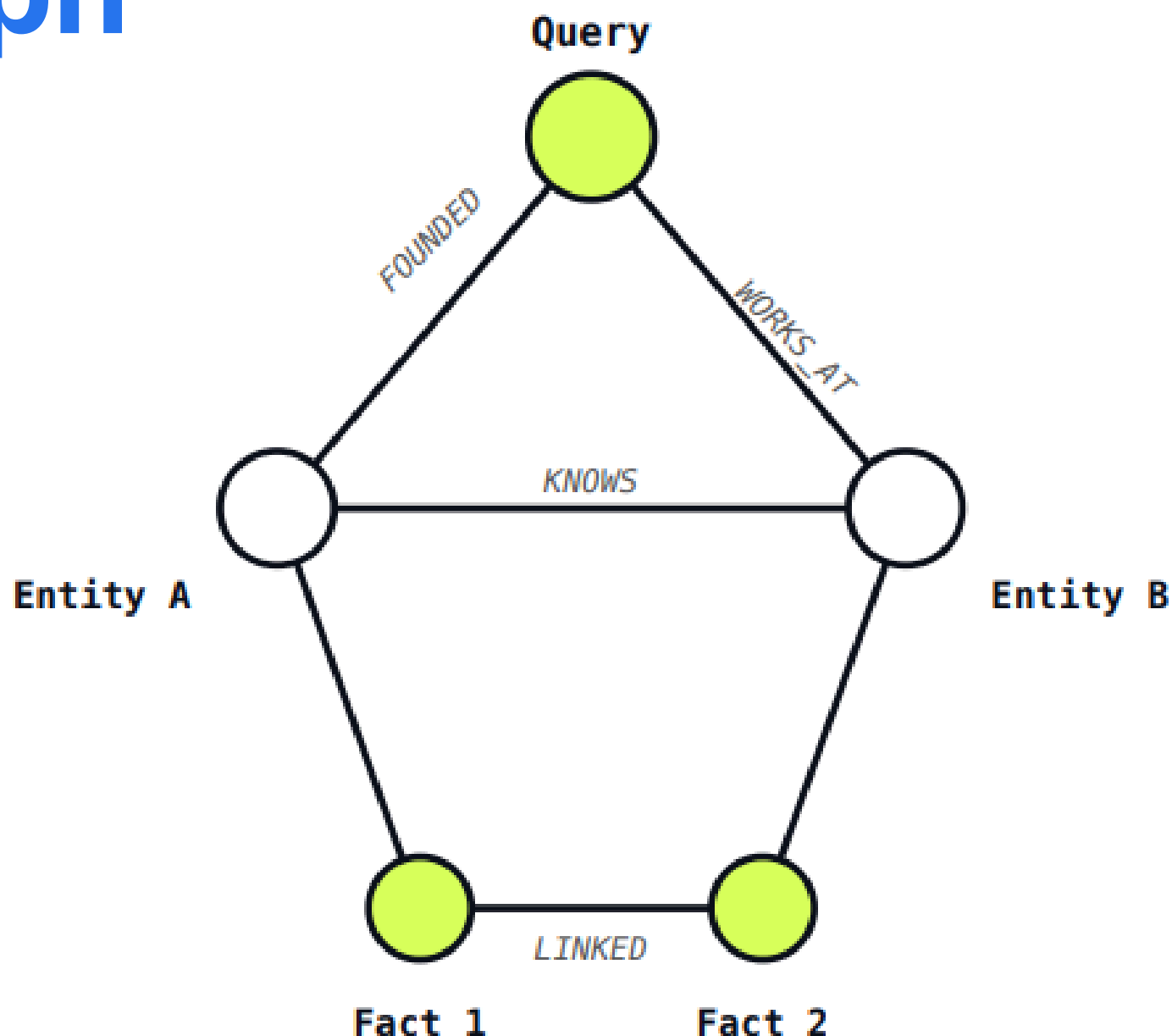
Plain RAG

- ✗ Retrieves isolated chunks by vector similarity
- ✗ Fails on multi-hop "A → B → C" questions
- ✗ Can't link entities across documents
- ✗ Shallow answers for complex queries
- ✗ No structure, no reasoning

Graph RAG

- ✓ Retrieves chunks plus structured relationships
- ✓ Traverses entity graphs for multi-hop reasoning
- ✓ Connects facts across sources via nodes & edges
- ✓ Richer, context-aware, explainable answers
- ✓ Built for enterprise-grade retrieval

Retrieval + a **knowledge graph**



A **Graph RAG system** stores your documents as entities and the relationships between them. When a query comes in, it pulls both the relevant text chunks and the connected entities giving the LLM structured context instead of a pile of passages.

The result?

Answers that reason across multiple facts, not just the nearest neighbor.

Step 1 of 7

Define your **entities & schema**

Before you touch a graph, decide what you're modeling. Identify the entity types (people, products, diseases, tickets) and the relationships that connect them. A clean schema is 80% of a useful graph.

 Domain docs

 Entity types

 Relation types

 Ontology

Step 2 of 7

Extract **entities & relationships.**

Run your raw documents through an LLM (or a specialized NER pipeline) to pull out triples: (subject, predicate, object). This is where unstructured text becomes structured knowledge.

Step 3 of 7

Build the **knowledge graph**.

Load your triples into a graph database. Every entity becomes a node, every relationship becomes an edge. You now have a queryable, traversable map of your domain.



Neo4j / NetworkX



Nodes



Edges



Properties

Step 4 of 7

Chunk & embed with graph context.

Don't embed chunks in isolation. Attach each chunk to its related nodes, then embed both the text and the surrounding graph neighborhood into a vector store. This is what makes retrieval graph-aware.

Step 5 of 7

Hybrid retrieval: vectors + graph walks.

On each query, do two retrievals in parallel: a semantic search over your vector store, and a graph traversal from the query's anchor entities. Merge both signals into a unified context.

 Vector search Graph walk Multi-hop Rerank

Step 6 of 7

Feed **graph context** to the LLM.

Pass the merged context text chunks plus the entity subgraph into your prompt. The LLM now has both the language and the logic to reason across multiple facts and produce grounded answers.

Step 7 of 7

Evaluate & iterate. Always.

Benchmark your Graph RAG against plain RAG on the same queries. Measure retrieval precision, multi-hop accuracy, and answer faithfulness then refine your schema, extraction, and traversal logic.

A full Graph RAG system - hands-on.

Foundations

Why RAG breaks on complex queries and how knowledge graphs fix it. Learn the components of a Graph RAG system.

Build a RAG baseline

Ship a traditional RAG pipeline on a real problem. See exactly where and why it falls short.

Build Graph RAG

Construct the knowledge graph, wire up hybrid retrieval, and compare outputs side-by-side with plain RAG.

Evaluate and ship

Measure Graph RAG performance with real evaluation techniques go beyond basic similarity scores.

[Enroll for Free](#)

Build **Graph RAG** systems that actually reason.

Intermediate level · Hands-on project · Certificate of completion ·

Graph RAG: Build Knowledge Graph Powered Retrieval Systems

Intermediate Level	128+ Students Enrolled	1 Hr 30 Mins Duration
------------------------------	----------------------------------	---------------------------------

[Enroll for Free](#)



Soumil Jain

